

PROJECT REPORT

Campus Lost and Found System

Build Your Own Project – VITyarthi

Student Name	Aditya Verma
Registration Number	25BAI10231
Technology	Java
Development Environment	IntelliJ IDEA
Version Control	Git and GitHub
Academic Year	2026

A simple Java application for reporting, finding and claiming lost and found items on a college campus.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Target Users
6. Functional Requirements
7. Non-Functional Requirements
8. System Architecture
9. Process Workflow
10. UML Design
11. Design Decisions and Rationale
12. Implementation Details
13. Screenshots and Results
14. Testing Approach
15. Challenges Faced
16. Learnings and Key Takeaways
17. Future Enhancements
18. Conclusion
19. References

Project structure

The source code is organized into a main package, a model package for data classes, and a service package for application logic. The project is maintained in a GitHub repository using Git.

1. INTRODUCTION

The Campus Lost and Found System is a Java-based console application made to handle lost and found items in a college campus. The idea came from a common campus problem: people often lose things, while someone else may find them but have no simple way to report them.

The application gives students and staff one place to report a lost item, report a found item, look through available items and submit a claim. An administrator can then check claims and approve or reject them. The program is intentionally simple, but it follows a clear structure so that it can be extended later.

The project also demonstrates practical use of object-oriented programming, inheritance, collections, input validation, basic authentication, modular design and Git version control.

2. PROBLEM STATEMENT

On a college campus, information about lost belongings is often shared through informal messages, class groups or notices. These methods are difficult to search and it is easy for information to get missed.

A small centralized application can make this process more organized. The proposed system records lost and found item details, allows searching, and gives users a way to submit claims. The administrator can then manage those claims instead of handling everything manually.

3. OBJECTIVES

- Provide a simple way to report lost items.
- Provide a simple way to report found items.
- Allow users to view all reported items.
- Allow searching by item name and category.
- Allow users to submit claims for an existing item.
- Allow an administrator to approve or reject claims.
- Show basic statistics for items and claims.
- Validate important user inputs and handle common errors.
- Keep the program modular and easy to understand.

4. SCOPE OF THE PROJECT

The current version focuses on the basic lost-and-found workflow inside a campus. It covers item reporting, searching, claiming and administrator review. The application is console based and stores information in memory using Java collections while the program is running.

The project does not currently include a permanent database, image uploads, email notifications or a web interface. These are possible improvements for a future version.

5. TARGET USERS

- **Students:** can report lost or found belongings, search items and submit claims.

- **Faculty and staff:** can use the same reporting and searching features.
- **Administrator:** can authenticate and manage submitted claims by approving or rejecting them.

6. FUNCTIONAL REQUIREMENTS

6.1 Report Lost Item

The user enters the item name, category, location and description. The application creates a `LostItem` object, gives it an Item ID and stores it with the status **LOST**.

6.2 Report Found Item

The user enters details of an item that has been found. The application creates a `FoundItem` object and stores it with the status **FOUND**.

6.3 View All Items

The application displays all available items with their ID, name, category, location, description and current status.

6.4 Search Item by Name

The user can search using part or all of an item name. The search is case-insensitive, which makes it easier to use from the console.

6.5 Search Item by Category

The user can search for items by category, such as books, electronics or bags.

6.6 Submit Claim

The user enters an Item ID and a reason for claiming it. Before accepting the claim, the program checks that the Item ID exists. A valid claim starts with the status **PENDING**.

6.7 View All Claims

The system displays each claim with its Claim ID, User ID, Item ID, reason and status.

6.8 Administrator Claim Management

The administrator must provide the configured username and password before approving or rejecting a claim. The claim status is changed to **APPROVED** or **REJECTED**.

6.9 Project Statistics

The application shows counts for total items, lost items, found items, total claims, pending claims, approved claims and rejected claims.

7. NON-FUNCTIONAL REQUIREMENTS

- **Usability:** the menu-driven interface keeps the basic operations easy to follow.
- **Reliability:** important inputs are checked before the system performs an operation.
- **Security:** administrator claim actions require username and password verification.
- **Maintainability:** model classes and service classes are separated into packages.

- **Error handling:** empty values, invalid Item IDs, incorrect admin credentials and invalid menu choices are handled with messages.
- **Efficiency:** ArrayList is used for straightforward in-memory storage and iteration.

8. SYSTEM ARCHITECTURE

The application uses a simple layered structure. Main.java acts as the user-facing part of the program. The model layer represents the entities, while the service layer contains the main operations. The current storage is in memory using ArrayList.

The main parts are:

- **Presentation layer:** Main.java handles the console menu, input and output.
- **Service layer:** ItemService, FoundItemService and ClaimService handle the application logic.
- **Model layer:** User, Admin, Item, LostItem, FoundItem and Claim represent the data used by the system.
- **Data storage:** ArrayList collections keep items and claims while the application is running.

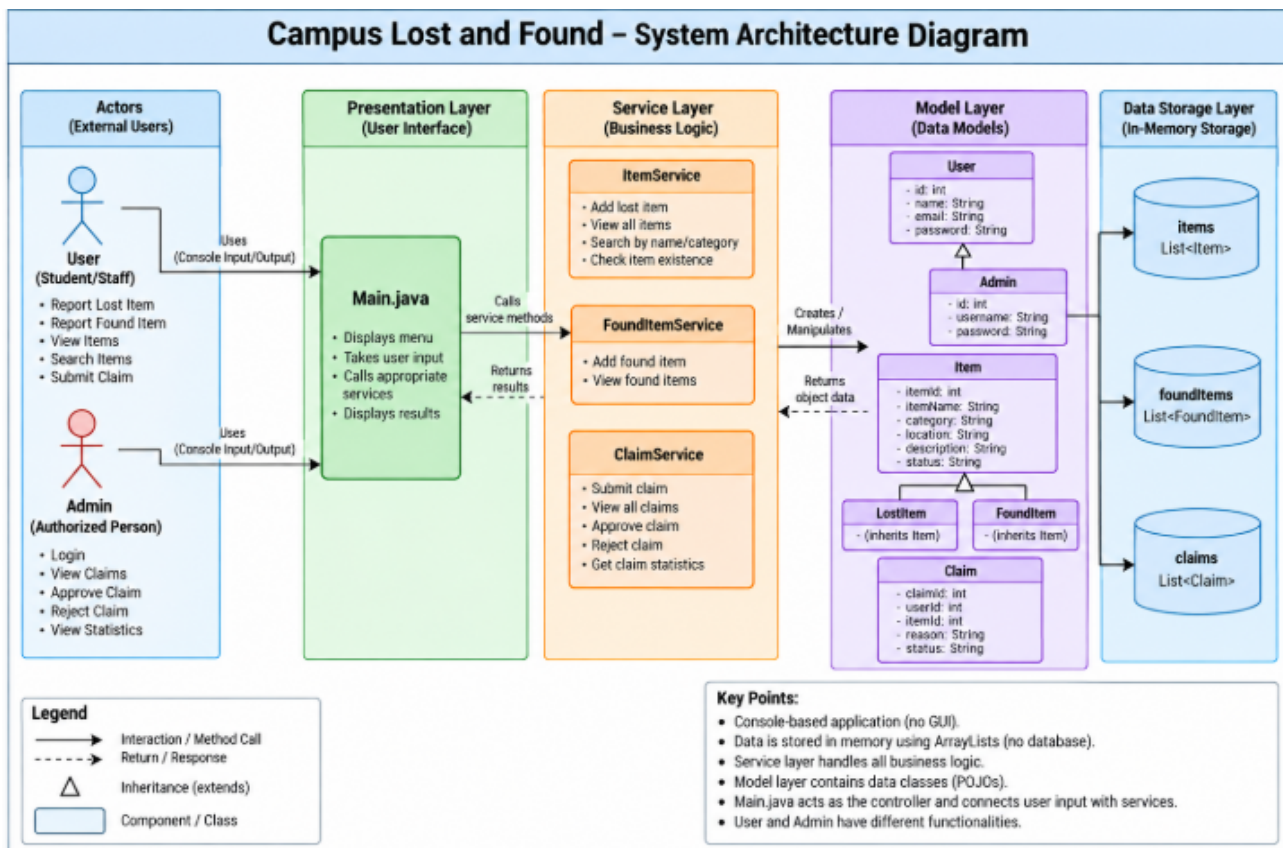


Figure 1: System Architecture Diagram

9. PROCESS WORKFLOW

The normal flow starts with the main menu. A user chooses an operation, enters the required details and receives a result. After completing an operation, the program returns to the main menu.

Administrator actions follow a separate authentication step.

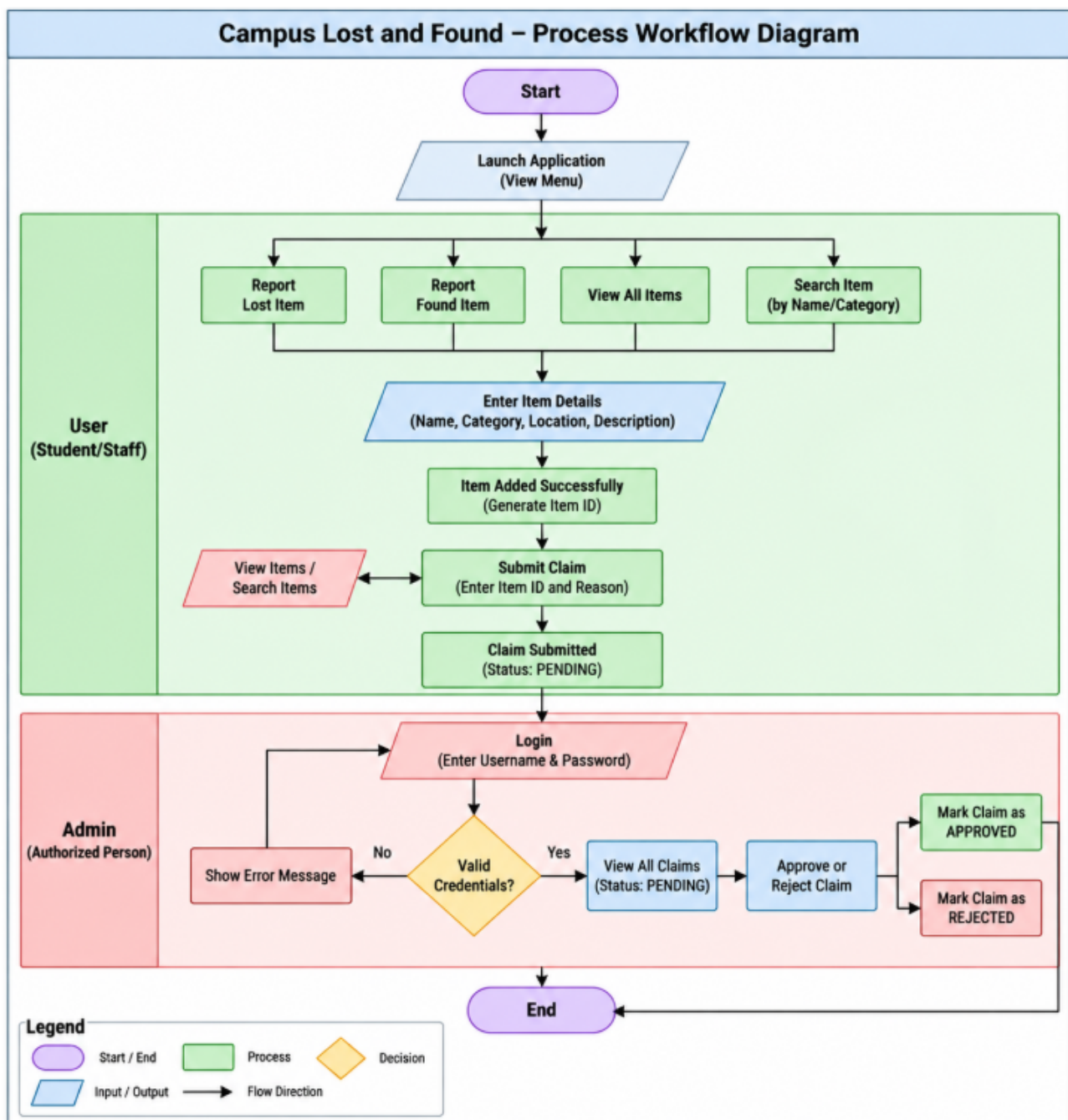


Figure 2: Process Workflow Diagram

10. UML DESIGN

UML diagrams were prepared to show the main actors, classes, interactions and overall structure of the application.

10.1 Use Case Diagram

The use case diagram shows the two main actors: User and Admin. Users handle item reporting, searching and claims. The administrator handles claim management after login.

Campus Lost and Found – Use Case Diagram

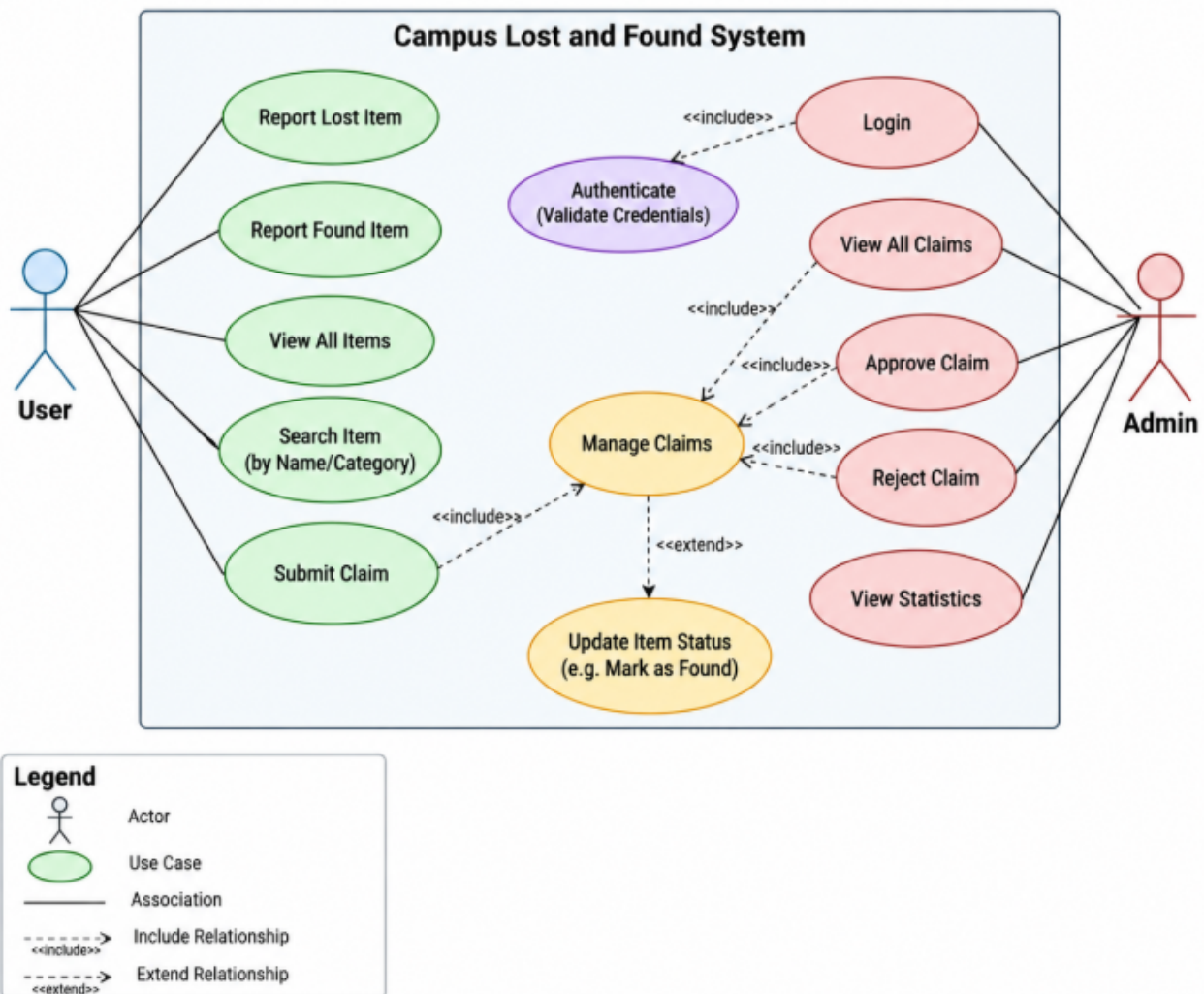


Figure 3: Use Case Diagram

10.2 Class Diagram

The class diagram shows the model and service classes used in the project. **LostItem** and **FoundItem** inherit common properties from **Item**. **Claim** stores the IDs needed to connect a claim with a user and an item.

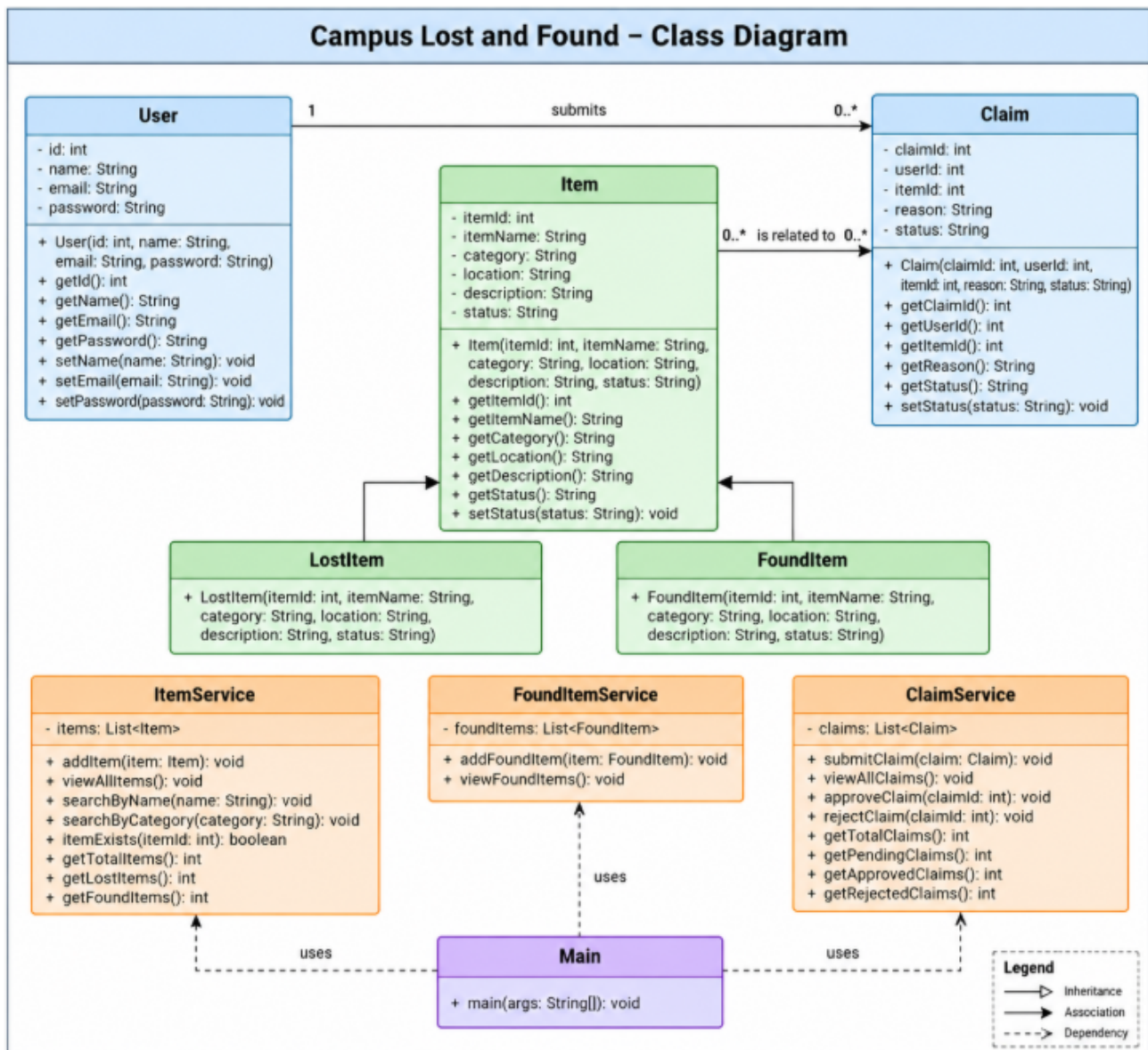


Figure 4: Class Diagram

10.3 Sequence Diagram

The sequence diagram focuses on the claim process. It shows how Main checks the Item ID through ItemService, creates/submits the claim through ClaimService, and how administrator actions update the claim status.

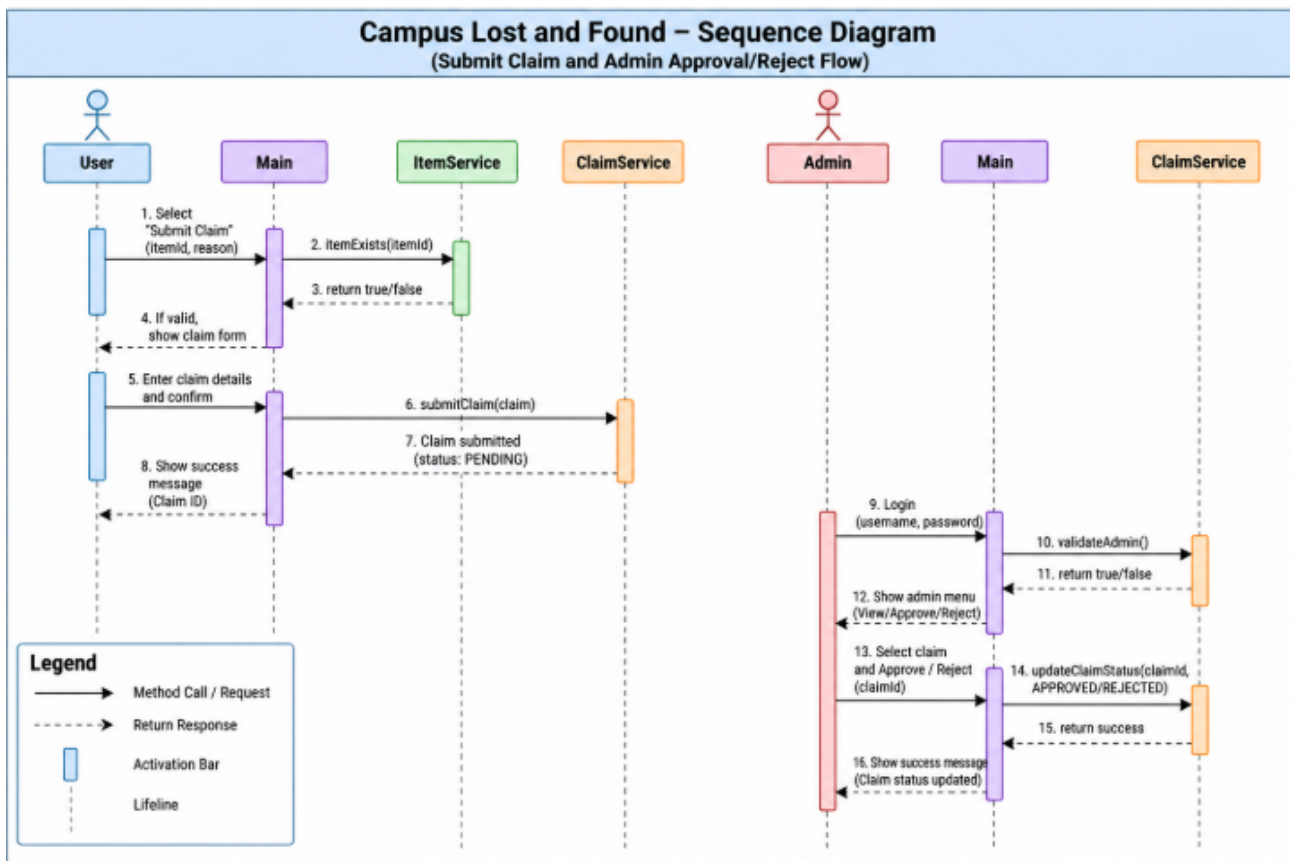


Figure 5: Sequence Diagram

11. DESIGN DECISIONS AND RATIONALE

Object-oriented classes were used because the project represents real entities such as users, items and claims. This makes the code easier to divide into logical parts.

Inheritance was used for `LostItem` and `FoundItem` because both are specialized forms of `Item` and share the same basic information. This avoids repeating the common item fields.

Service classes were introduced so that operations are not all placed inside `Main.java`. Item-related operations are handled by `ItemService`, found-item operations by `FoundItemService`, and claim operations by `ClaimService`.

`ArrayList` was chosen for the current version because the project is a small console application and does not yet require permanent storage. A database can be added later without changing the basic idea of the model classes.

Validation was added at the points where incorrect data could cause a problem, such as empty names or an Item ID that does not exist.

12. IMPLEMENTATION DETAILS

12.1 Model Classes

User.java: stores user ID, name, email and password.

Admin.java: stores administrator ID, username and password.

Item.java: stores common item information such as ID, name, category, location, description and status.

LostItem.java: extends Item and represents a lost item.

FoundItem.java: extends Item and represents a found item.

Claim.java: stores claim ID, user ID, item ID, reason and claim status.

12.2 Service Classes

ItemService.java: adds, displays and searches items and calculates item statistics.

FoundItemService.java: stores and displays found items.

ClaimService.java: submits, displays, approves and rejects claims and calculates claim statistics.

12.3 Main Class

Main.java controls the complete menu-driven workflow. Scanner is used to read input from the console. The application assigns item IDs starting from 101 and claim IDs starting from 1 during a program run.

12.4 Project Structure

CampusLostAndFound	
src	
com.lostfound	Main.java
model	User, Admin, Item, LostItem, FoundItem, Claim
service	ItemService, FoundItemService, ClaimService
README.md	Project overview and usage information
statement.md	Problem statement, scope and high-level features

13. SCREENSHOTS AND RESULTS

The following screenshots were taken while running the application in IntelliJ IDEA. They show the main menu, item reporting and the claim workflow. Six screenshots are included so that the important working parts of the project can be seen without making the report unnecessarily long.

13.1 Main Menu

The main menu lists all the major operations available to the user.

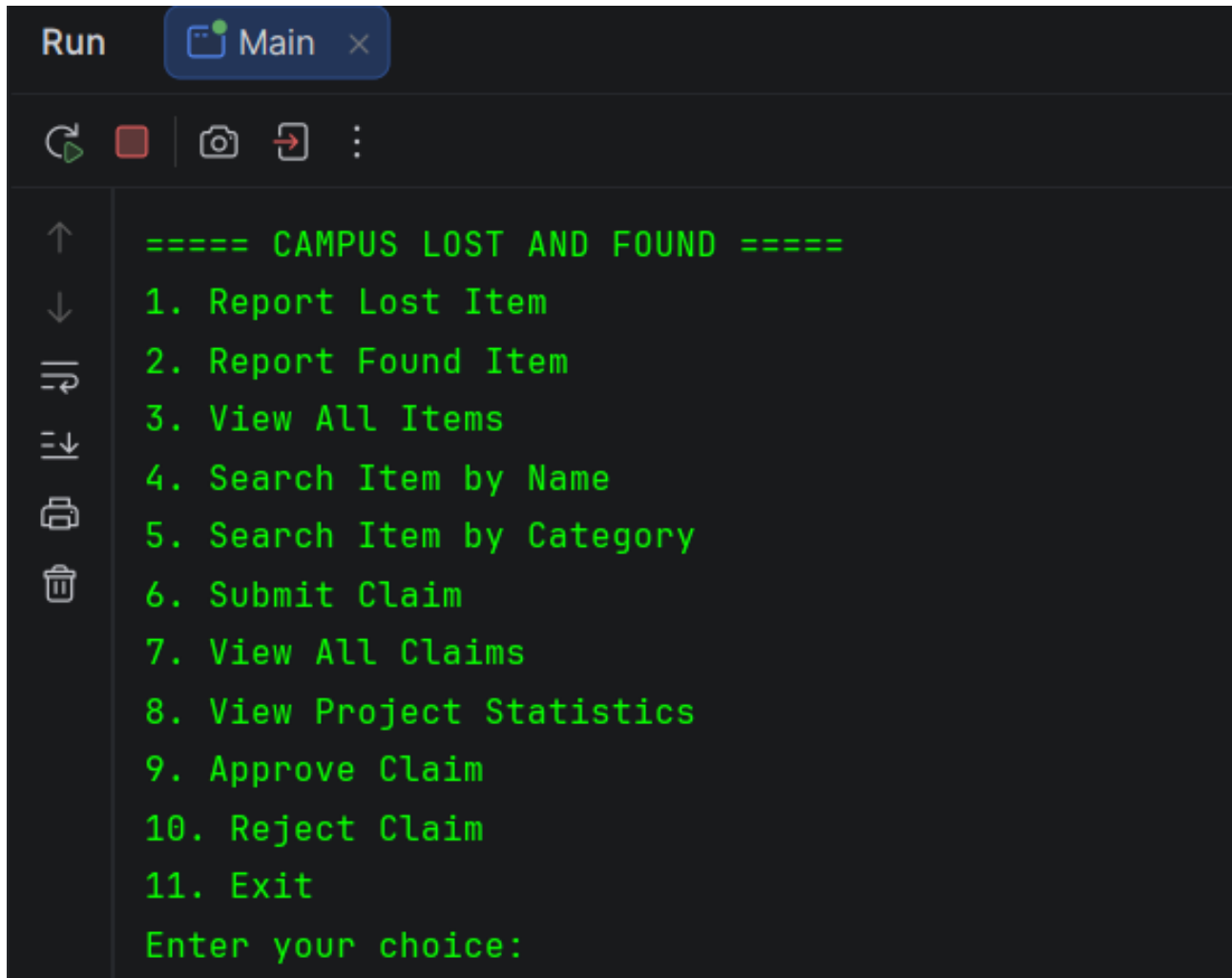
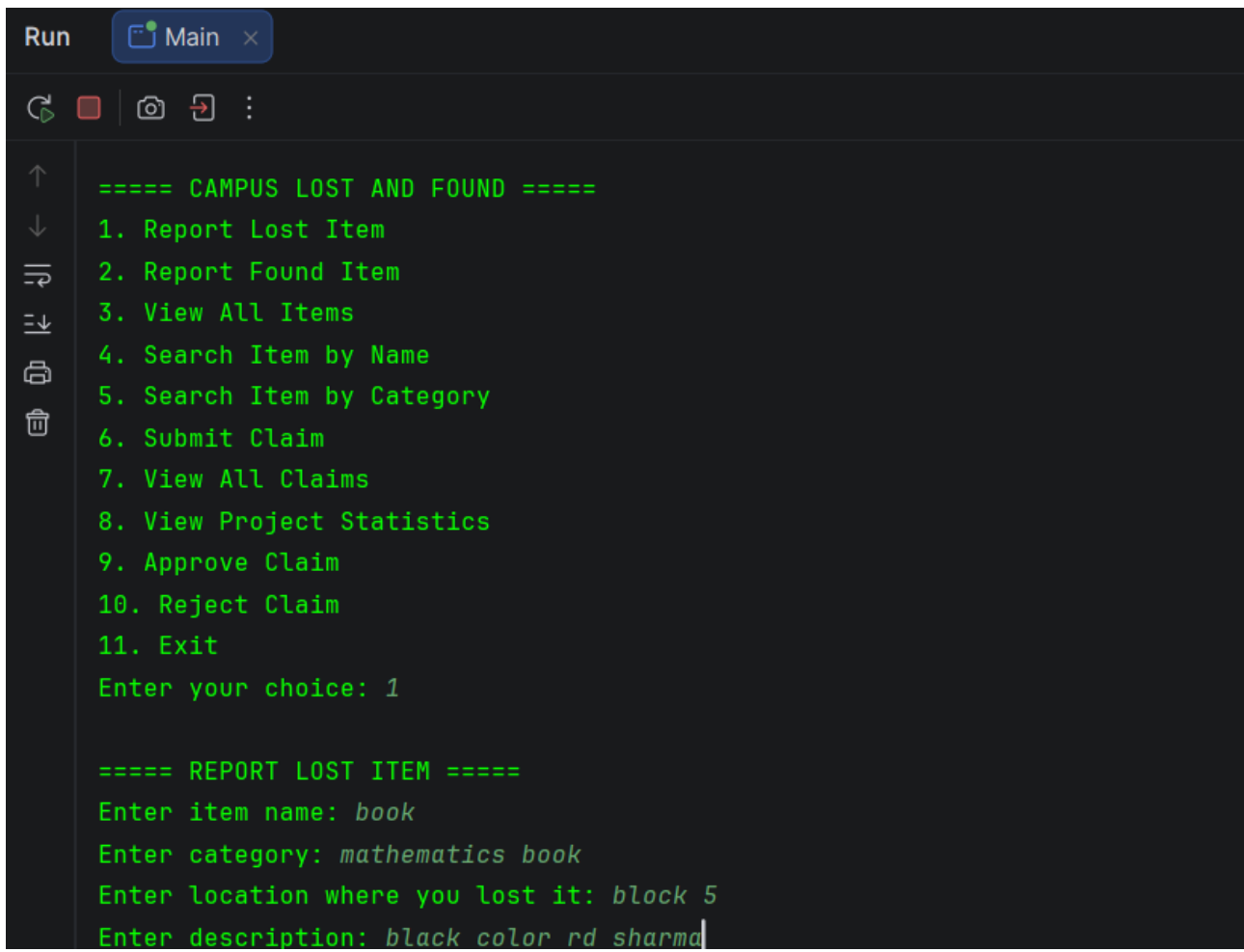


Figure 6: Main Menu

13.2 Entering a Lost Item

This screenshot shows the user entering the item name, category, location and description for a lost item.



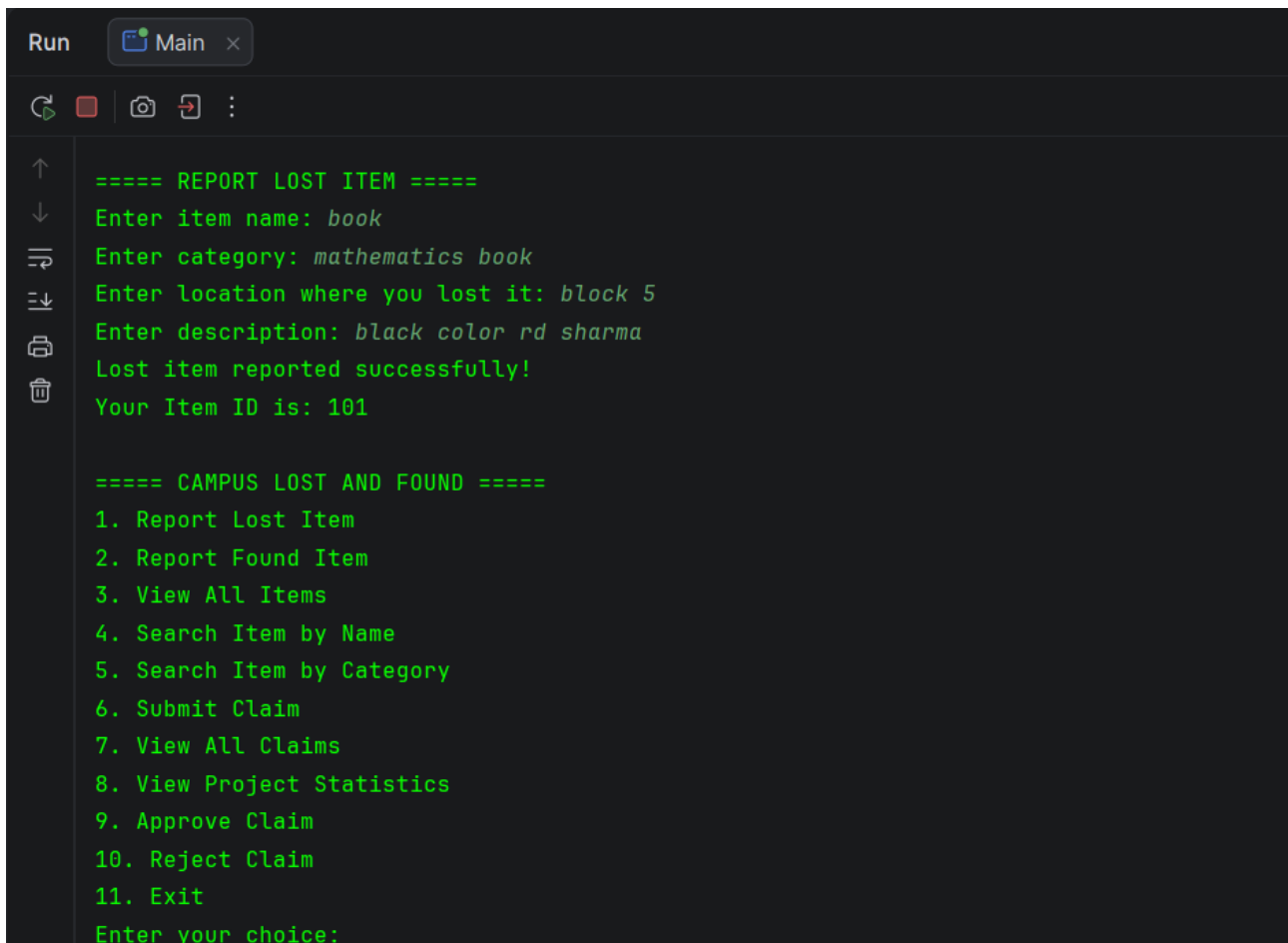
```
Run Main x
===== CAMPUS LOST AND FOUND =====
1. Report Lost Item
2. Report Found Item
3. View All Items
4. Search Item by Name
5. Search Item by Category
6. Submit Claim
7. View All Claims
8. View Project Statistics
9. Approve Claim
10. Reject Claim
11. Exit
Enter your choice: 1

===== REPORT LOST ITEM =====
Enter item name: book
Enter category: mathematics book
Enter location where you lost it: block 5
Enter description: black color rd sharma
```

Figure 7: Lost Item Input

13.3 Lost Item Reported Successfully

After valid details are entered, the system confirms the report and displays the generated Item ID.



The screenshot shows a terminal window with a dark background and green text. At the top, there's a tab labeled 'Main' with a close button. Below the tab is a toolbar with icons for running, stopping, and other actions. The main area of the terminal displays the following text:

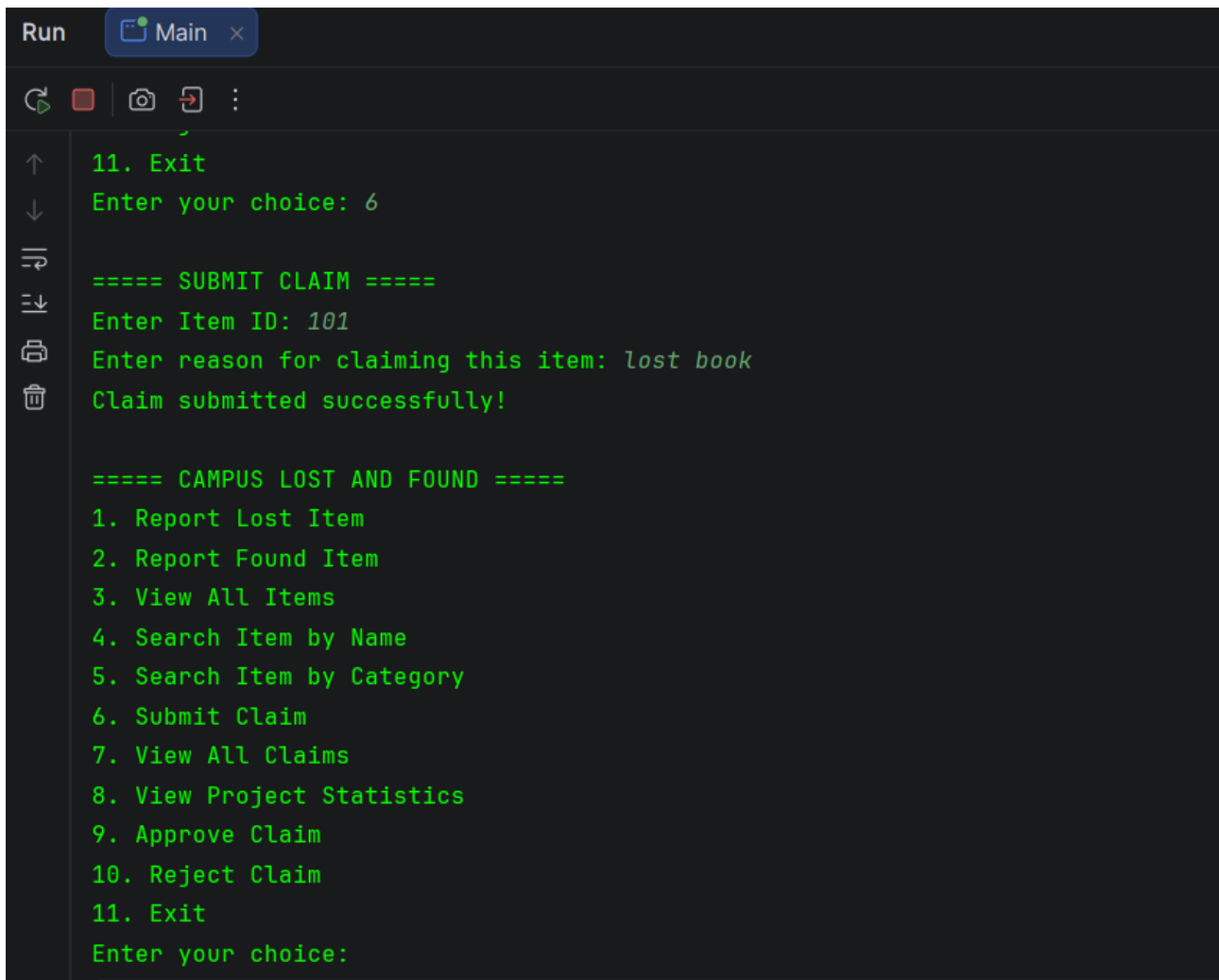
```
==== REPORT LOST ITEM ====
Enter item name: book
Enter category: mathematics book
Enter location where you lost it: block 5
Enter description: black color rd sharma
Lost item reported successfully!
Your Item ID is: 101

==== CAMPUS LOST AND FOUND ====
1. Report Lost Item
2. Report Found Item
3. View All Items
4. Search Item by Name
5. Search Item by Category
6. Submit Claim
7. View All Claims
8. View Project Statistics
9. Approve Claim
10. Reject Claim
11. Exit
Enter your choice:
```

Figure 8: Successful Lost Item Report

13.4 Submitting a Claim

Here, the user enters an existing Item ID and a reason for claiming it. The system accepts the claim and confirms the submission.



The screenshot shows a Java IDE with a 'Run' button and a 'Main' tab. The console output is as follows:

```
11. Exit
Enter your choice: 6

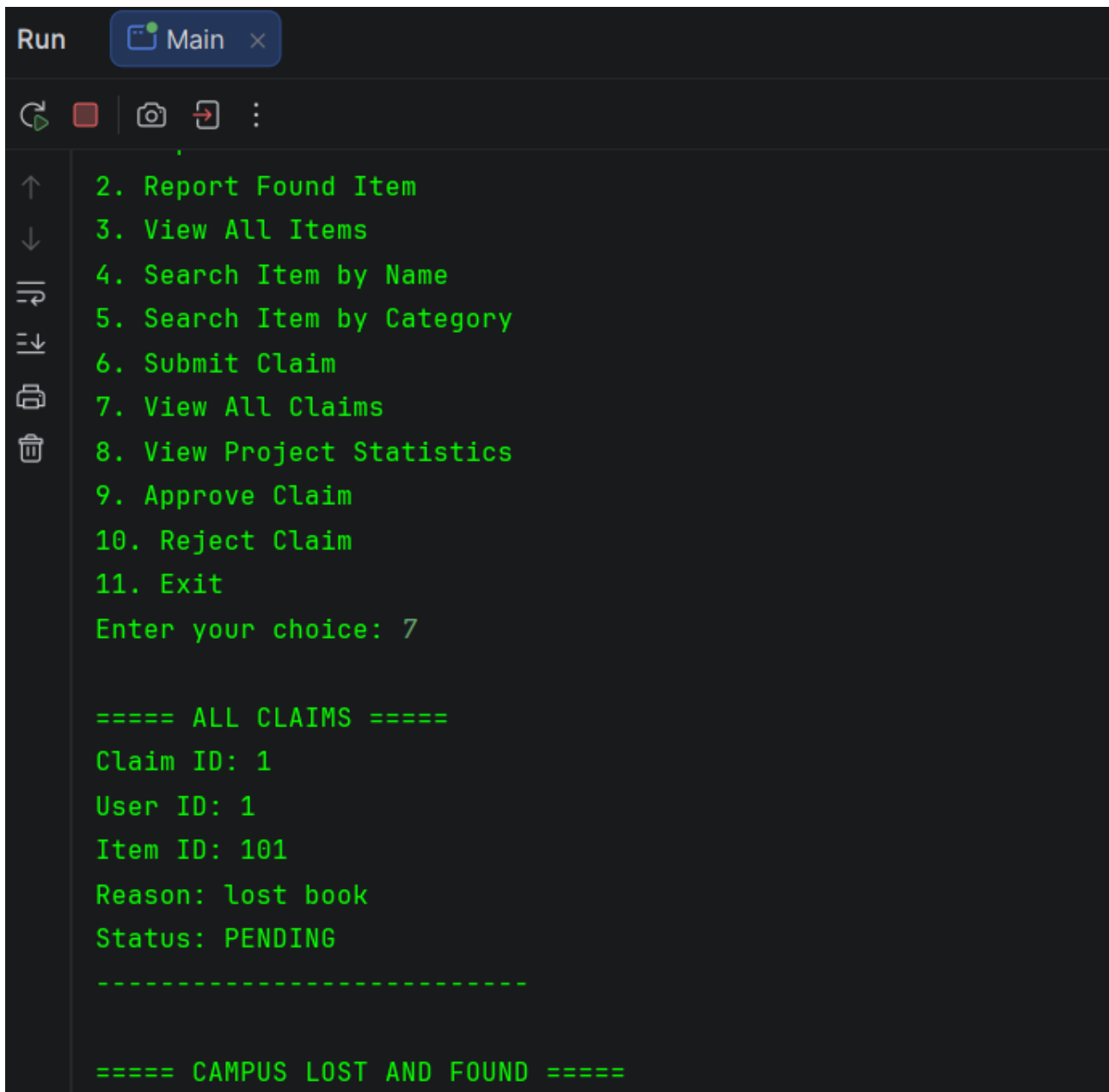
==== SUBMIT CLAIM ====
Enter Item ID: 101
Enter reason for claiming this item: lost book
Claim submitted successfully!

==== CAMPUS LOST AND FOUND ====
1. Report Lost Item
2. Report Found Item
3. View All Items
4. Search Item by Name
5. Search Item by Category
6. Submit Claim
7. View All Claims
8. View Project Statistics
9. Approve Claim
10. Reject Claim
11. Exit
Enter your choice:
```

Figure 9: Claim Submission

13.5 Viewing Claims

The claims screen shows the claim ID, user ID, item ID, reason and current status. In this example, the new claim is shown as PENDING.



```
Run Main x
[Icons: Run, Stop, Copy, Paste, etc.]

↑ 2. Report Found Item
↓ 3. View All Items
↕ 4. Search Item by Name
↕ 5. Search Item by Category
↕ 6. Submit Claim
🖨 7. View All Claims
🗑 8. View Project Statistics
9. Approve Claim
10. Reject Claim
11. Exit
Enter your choice: 7

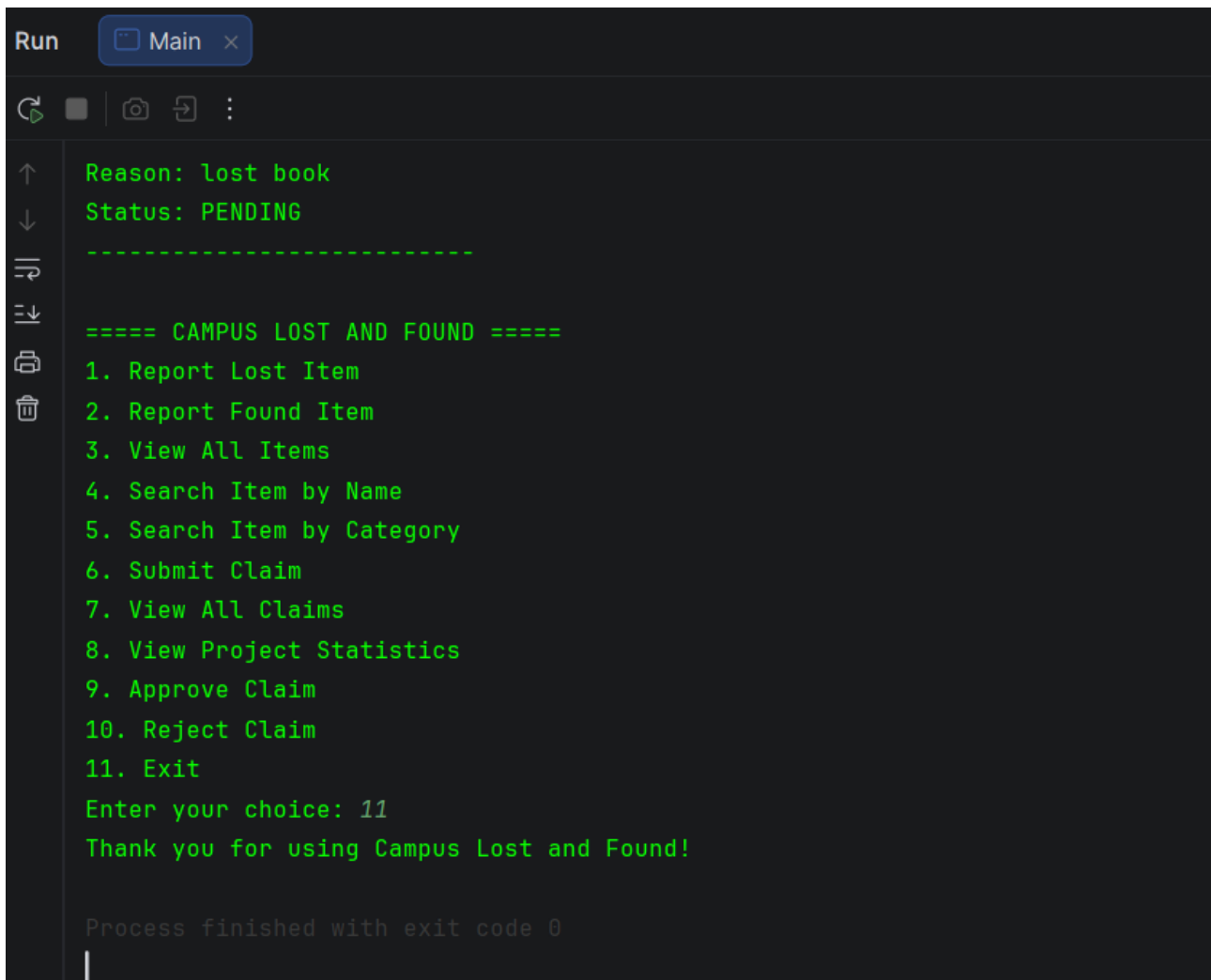
===== ALL CLAIMS =====
Claim ID: 1
User ID: 1
Item ID: 101
Reason: lost book
Status: PENDING
-----

===== CAMPUS LOST AND FOUND =====
```

Figure 10: View All Claims

13.6 Exiting the Application

The final screenshot shows the program returning the exit message and finishing normally.



```
Run Main x
Reason: lost book
Status: PENDING
-----

==== CAMPUS LOST AND FOUND ====
1. Report Lost Item
2. Report Found Item
3. View All Items
4. Search Item by Name
5. Search Item by Category
6. Submit Claim
7. View All Claims
8. View Project Statistics
9. Approve Claim
10. Reject Claim
11. Exit
Enter your choice: 11
Thank you for using Campus Lost and Found!

Process finished with exit code 0
```

Figure 11: Application Exit

14. TESTING APPROACH

Testing was carried out by running the program from IntelliJ IDEA and checking the result of each major operation. Both normal cases and a few invalid-input cases were tested.

Test	Action	Expected Result	Result
1	Report lost item	Item created with LOST status	Pass
2	Report found item	Item created with FOUND status	Pass
3	View all items	Reported items are displayed	Pass
4	Search by name	Matching item is displayed	Pass
5	Search by category	Matching items are displayed	Pass
6	Submit valid claim	Claim created as PENDING	Pass
7	View claims	Claim details are displayed	Pass
8	View statistics	Item and claim counts displayed	Pass
9	Approve claim	Claim becomes APPROVED	Pass
10	Reject claim	Claim becomes REJECTED	Pass
11	Wrong admin password	Access denied	Pass

Test	Action	Expected Result	Result
12	Invalid Item ID	Claim is not accepted	Pass
13	Empty required input	Validation message shown	Pass
14	Exit	Program finishes normally	Pass

The main functions and the tested error cases worked as expected during the test run.

15. CHALLENGES FACED

- Keeping a multi-class Java project organized was initially challenging, so model and service packages were used.
- Lost and found items have many common fields, so inheritance was used to avoid repeating code.
- Claim handling needed clear states, so PENDING, APPROVED and REJECTED were used.
- The application needed to respond properly to empty input and invalid IDs, so validation checks were added.
- Learning the Git workflow was another part of the project. The local repository was initialized, committed and connected to GitHub, and the final project was pushed to the main branch.

16. LEARNINGS AND KEY TAKEAWAYS

- How to turn a small real-world problem into a working software project.
- How classes and objects can represent real-world entities.
- How inheritance can reduce repeated code.
- How ArrayList can be used to manage a collection of objects.
- Why separating business logic into service classes makes a project easier to manage.
- How input validation improves the reliability of a console application.
- How UML diagrams help explain a software design before or alongside implementation.
- How to use Git and GitHub to keep a project under version control.

17. FUTURE ENHANCEMENTS

- Connect the application to MySQL or another database for permanent storage.
- Build a graphical user interface so that users do not need to use the console.
- Convert the project into a web-based campus portal.
- Add complete student/staff registration and login.
- Allow users to upload photos of lost or found items.
- Add automatic matching between lost and found items using name, category and location.
- Add notifications when a possible match is found or a claim status changes.

18. CONCLUSION

The Campus Lost and Found System provides a practical way to organize lost and found items on a college campus. It brings together item reporting, searching and claim management in one small application.

The project also provided hands-on practice with Java object-oriented programming, inheritance, collections, modular design, validation, testing and GitHub. Although the current version is a console application with in-memory storage, its structure gives a clear starting point for future improvements such as database support, a graphical interface and a web version.

Overall, the project meets its main goal of providing a simple and usable lost-and-found workflow while demonstrating the software development concepts covered in the project.

19. REFERENCES

1. VITyarthi, *Build Your Own Project – General Project Instructions & Submission Guidelines*.
2. Java programming documentation and learning resources used during development.
3. Git and GitHub documentation used for repository and version-control setup.
4. IntelliJ IDEA documentation and development environment resources.

END OF REPORT